

Arquitetura de banco de dados

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Reconhecer as arquiteturas e os esquemas de banco de dados.
- Descrever os tipos de bancos de dados e seus usuários.
- Comparar as características dos principais bancos de dados relacionais.

Introdução

Os bancos de dados surgiram principalmente em função da grande necessidade de armazenar dados de forma permanente e organizada. É importante salientar que o volume de dados que tem surgido é cada vez maior. Graças ao advento dos sistemas de gerenciamento de banco de dados (SGBDs), é possível extrair diversas informações que auxiliem no processo de tomada de decisão, exatamente no formato de que precisamos e em poucos minutos.

Além disso, a arquitetura de um banco de dados determina a escolha da configuração adequada para a implantação de um SGBD e as regras para a estruturação dos dados a partir da abstração do mundo real, tudo isso de acordo com o projeto de banco de dados. A arquitetura de um banco de dados sofre grande influência do sistema de computadores no qual um SGBD está instalado. Logo, saber mensurar a capacidade de processamento exigida por um SGBD irá ser determinante para a escolha da arquitetura mais adequada em um projeto.

No geral, os SGBDs oferecem a possibilidade de armazenar e manipular arquivos. Alguns permitem trabalhar com diferentes arquiteturas/estruturas, e cada arquitetura apresenta vantagens e desvantagens quando adotada em certo contexto e com determinada aplicação.

Neste capítulo, você vai estudar as diferentes arquiteturas de sistemas de banco de dados. Você também vai conferir os diferentes modelos de sistemas, bem como a sua evolução histórica, as suas principais funções, os seus recursos e as suas finalidades. Por fim, você vai explorar as características e diferenças entre os principais bancos de dados relacionais.

1 Classificação dos sistemas de gerenciamento de banco de dados

Um **banco de dados**, segundo Elmasri e Navathe (2011), é uma coleção de dados relacionados que representam aspectos do mundo real (minimundo ou universo de discurso). Mudanças no mundo real devem ser refletidas nesse ambiente.

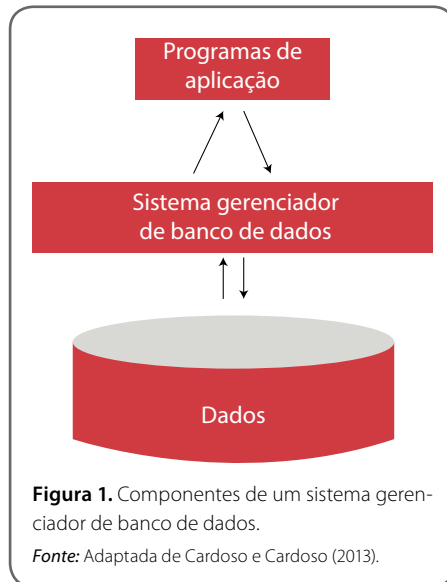


Saiba mais

Para Machado (2014, p. 18), **minimundo** é considerado uma:

[...] porção da realidade captada pelo analista, que a gestão de negócios de uma organização tem interesse em observar, controlar e administrar. A complexidade existente no momento de analisar um minimundo pode levar o analista a subdividi-lo em partes menores, às quais damos o nome de visão de processo de negócio (MACHADO, 2014, p. 18).

Os SGBDs são constituídos por um conjunto de *software* com diferentes funcionalidades, que se complementam para oferecer serviços de criação e manipulação de bancos de dados, segundo Elmasri e Navathe (2011). Um SGBD é um sistema de *software* de uso geral, que facilita o processo de definição, construção, manipulação e compartilhamento de dados, simplificando a experiência do usuário. A Figura 1 mostra a composição de um SGBD.



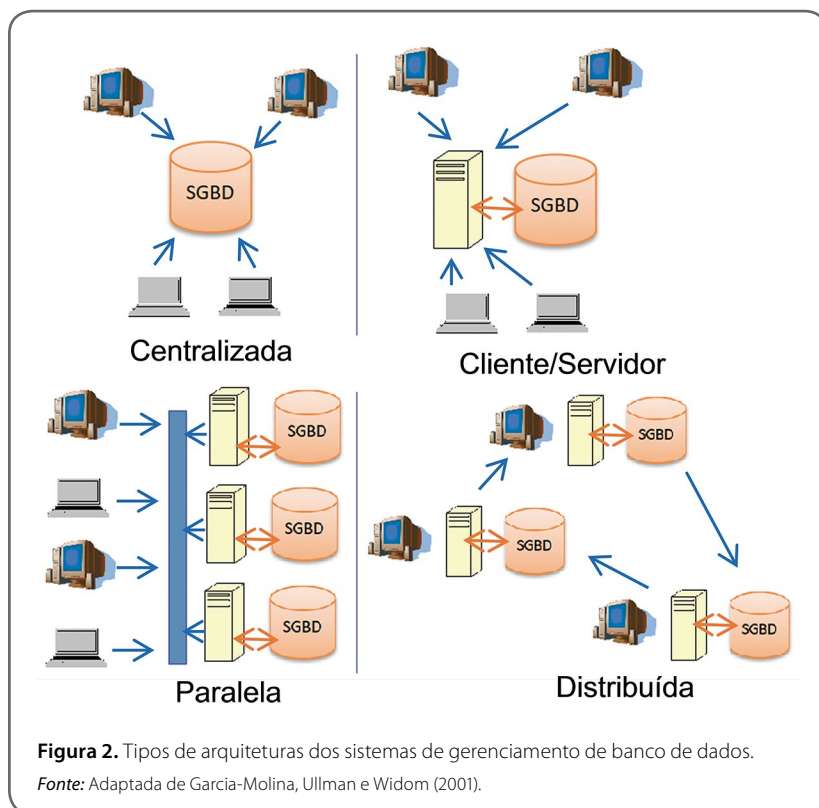
O SGBD oferece recursos para **manipulação dos dados** armazenados, os quais permitem inserção, alteração e exclusão de dados. Além disso, o sistema disponibiliza consultas estruturadas aos dados, recurso amplamente utilizado na criação de relatórios. Outro mecanismo importante dos SGBDs é o **compartilhamento de dados**, que dá aos usuários a possibilidade de acessar simultaneamente o banco. Ou seja, o sistema controla os acessos e protege a integridade dos dados.

As consultas são feitas por meio de um **programa de aplicação**, utilizado pelo usuário para digitar, solicitar uma consulta e exibir o seu retorno. O programa de aplicação pode ser qualquer *software* que solicite consultas de dados a um determinado banco de dados. Nesse caso, esse programa requisita conexão ao SGBD, enviando o usuário e a senha para autenticação. Após aberta a conexão, o programa pode solicitar a consulta aos dados. Assim, o SGBD é responsável por executar as consultas e devolver os dados resultantes delas ao programa de aplicação. Os SGBDs também possuem recursos que buscam minimizar a perda de dados em caso de problemas com *software* e/ou *hardware*. São artifícios contra falhas inesperadas, os quais tentam evitar que os arquivos sejam corrompidos.

É possível classificar os SGBDs quanto à arquitetura de dados, quanto ao modelo de dados que utilizam e quanto aos tipos de dados que suportam. A seguir, vamos estudar essas classificações.

Arquitetura de dados

A **arquitetura** de um banco de dados sofre grande influência da arquitetura ou do sistema de computadores no qual o SGBD está instalado. Os modelos arquiteturais mais comuns e que estão relacionados ao esquema do ambiente no qual o banco de dados é instalado são do tipo centralizados, cliente/servidor, paralelos e distribuídos (GARCIA-MOLINA; ULLMAN; WIDOM, 2001), conforme mostra a Figura 2.



Cada arquitetura contém um conjunto próprio de características, como pode ser observado a seguir (GARCIA-MOLINA; ULLMAN; WIDOM, 2001).

- **Centralizada:** oferece processamento principal para todas as tarefas do sistema, incluindo os programas e aplicativos do usuário, o SGBD e o próprio banco de dados. O principal motivo para a utilização desse tipo de arquitetura ocorre por razão financeira, quando não existem recursos financeiros suficientes para a aquisição de um conjunto adequado de computadores com boa capacidade de processamento.
- **Cliente/servidor:** também conhecida como arquitetura em camadas, divide as responsabilidades do sistema entre as máquinas cliente e a(s) máquina(s) servidor. Assim, basta que outros computadores possuam uma interface para comunicação com o SGBD que está instalado no computador servidor. Os computadores que dão acesso ao banco de dados são chamados de clientes, o que origina o termo “cliente/servidor”. A arquitetura cliente/servidor ainda possibilita a exploração da capacidade de processamento dos computadores clientes para a realização de tarefas simples, o que reduz os custos de processamento dos computadores servidores.



Exemplo

Em uma arquitetura de três camadas, por exemplo:

- o servidor de banco de dados fica responsável pelo SGBD e pelo próprio banco de dados;
- o servidor de aplicação fica responsável pelas classes e códigos relacionados às regras de negócio da aplicação e pelo acesso ao SGBD para recuperação e, se necessário, pré-processamento dos dados, antes de entregá-los aos clientes; e
- as máquinas cliente ficam responsáveis pelas classes de interface e pequenos processamentos pertinentes ao cliente.

- **Paralela:** apresenta uma estrutura hierárquica, na qual um computador fica responsável pelo gerenciamento dos recursos, que são compartilhados por todos os dispositivos ligados à rede, ou seja, ligados em paralelo. Logicamente, assemelha-se à arquitetura centralizada; entretanto, fisicamente, há diversos computadores ligados em paralelo, compartilhando poder de processamento, entrada/saída, armazenamento e memória.
- **Distribuída:** os bancos de dados estão espalhados pela rede, assim como os componentes dos programas aplicativos que fazem uso de tais dados.

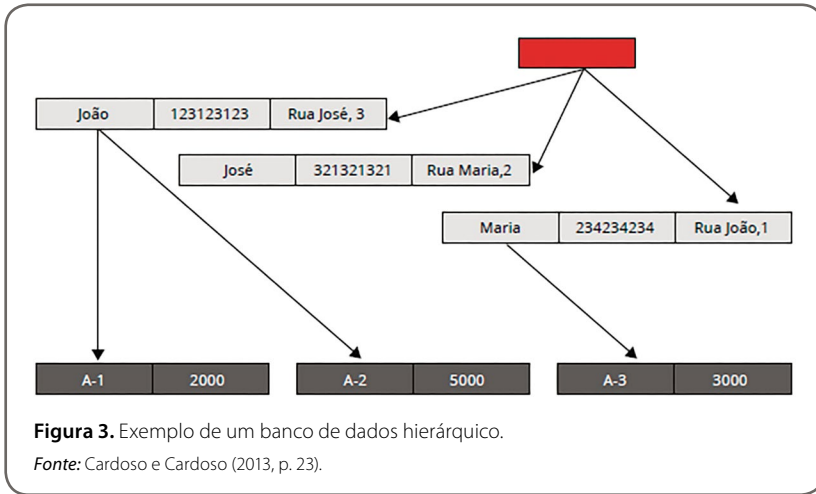
Diversas outras arquiteturas foram propostas, e os avanços da tecnologia permitem que novas possibilidades sejam vislumbradas. Além da arquitetura do sistema computacional, outro fator que pode influenciar na definição da estrutura arquitetural do SGBD é a estrutura organizacional. O ideal é implementar o esquema arquitetural que melhor se adeque às necessidades do cliente.

Modelo de dados

Quando falamos dos modelos de dados de um SGBD, estamos nos referindo à estrutura utilizada para organizar os dados no banco. Esse aspecto dos SGBDs, em específico, tem se modificado ao longo dos anos, com o surgimento dos SGBDs com modelos não relacionais (NoSQL), que propõem estruturas completamente diferentes para armazenamento dos dados. Existem várias estruturas de dados que podem ser adotadas por um SGBD. A seguir, serão apresentados alguns modelos de SGBDs existentes.

Modelo de banco de dados hierárquico

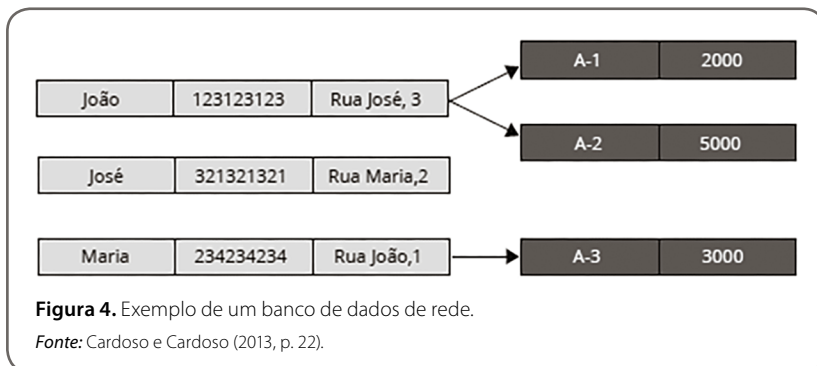
O **banco de dados hierárquico** é o primeiro tipo de banco de dados de que se tem notícia. Esse modelo foi desenvolvido por causa da consolidação dos discos endereçáveis; em função dessa característica, a organização de endereços físicos do disco é utilizada na sua estrutura (ALVES, 2013). A Figura 3 traz um exemplo de banco de dados hierárquico.



Segundo Alves (2013), em sistemas de banco de dados hierárquicos, são encontrados dois conceitos fundamentais: registros e relacionamentos pai-filho. O **registro** é uma coleção de valores que representam informações sobre uma dada entidade de um relacionamento. Quando se tem registros do mesmo tipo, denominam-se **tipos de registros**, similares às tabelas/relações do sistema relacional. Os registros que antecedem outros na hierarquia têm a denominação de **pai**, e os registros que os sucedem são chamados de **filhos**.

Modelo de banco de dados de rede

Os sistemas de **banco de dados de rede** são conhecidos como CODASYL, ou sistemas DBTG, pelo fato de terem sido definidos pelo grupo de tarefa de base de dados (*data base task group*) do comitê da Conference on Data Systems Languages (Conferência sobre Linguagens de Sistemas de Dados). O DBTG publicou, em 1971, um relatório que “[...] descrevia o modelo e a linguagem para utilização em bases de dados, embora esse relatório não definisse a organização dos dados propriamente ditos” (ALVES, 2013, p. 22). A Figura 4 ilustra um exemplo de banco de dados de rede.



Conforme leciona Alves (2013, p. 22):

[...] as duas estruturas fundamentais de um banco de dados de rede são os registros (*records*) e os conjuntos (*sets*). Os registros contêm dados relacionados e são agrupados em tipos de registros que armazenam os mesmos tipos de informações (como ocorre no sistema hierárquico).

Modelo de banco de dados relacional

É a estrutura mais comum aos SGBDs, e os SGBDs que a adotam são também os mais utilizados. Temos como exemplos Oracle, Microsoft SQL Server, MariaDB, MySQL, PostgreSQL etc. O **modelo de dados relacional** foi criado em 1985 por Edgar Frank Codd, que definiu que os bancos desse modelo devem apresentar a informação uma única vez, ou seja, devem possuir um identificador de cada informação para que não haja registros duplicados. Além disso, os dados precisam estar dispostos em forma de tabela. Ao todo, foram estabelecidas premissas para o modelo relacional, das quais podemos destacar:

- relações de integridade;
- recursos para inserção, alteração e remoção de dado em alto nível;
- acesso ao dado por meio da consulta de sua tabela.

A arquitetura das tabelas é dada pelo estabelecimento de relações e pelo uso de tuplas e atributos. Portanto, nos bancos de dados do modelo relacional, os dados são sempre salvos em tabelas, que nada mais são do que uma estrutura de linhas e colunas.

Tabelas

Quando falamos que as **tabelas** são relações, estamos dizendo que as lógicas entre os dados do banco são definições da tabela. Ao criar e modificar uma tabela, podemos estipular que determinados atributos se relacionam com dados de outra tabela. A construção dessas relações é fundamental para que quem precisar utilizar o banco possa executar consultas posteriores e compreender a lógica entre os dados.

Registros

Temos ainda os **registros**, também conhecidos como linhas ou tuplas. Todas as linhas de tabela são referências aos atributos definidos pelas colunas. É permitido que as linhas aceitem atributos com valores nulos, ou vazios, desde que isso tenha sido definido na estrutura da tabela, ou seja, que esteja especificado nos metadados.

Atributos

Por último, temos as **colunas** ou atributos, que são as identificações dos dados. Os atributos são responsáveis por caracterizar os dados inseridos em determinada coluna.

Modelo de banco de dados orientado a objetos

Os **bancos de dados orientados a objetos** trabalham com o armazenamento de dados na forma de objetos. Nesse caso, a definição de objetos é similar ao conceito utilizado nas linguagens de programação, como Java. Esse modelo é adotado quando os modelos de dados se tornam extremamente complexos para manipulação por meio de relações no modelo relacional. Assim, o armazenamento em objetos facilita o acesso e o uso dos dados quando o *software* trabalha com alguma linguagem orientada a objetos, evitando o consumo de tempo na conversão de tuplas em objetos, ao se utilizar um banco de dados relacional e uma linguagem orientada a objetos. Os dados ou objetos só podem ser acessados pelos métodos predefinidos para eles, e um objeto pode conter referências a outros objetos.

Modelo de banco de dados objeto-relacional

O **modelo de dados objeto-relacional** identifica os SGBDs que mesclam as características do modelo relacional e do orientado a objetos. Em resumo, armazena os dados conforme o modelo relacional, com tabelas estruturadas, mas possibilita a conversão imediata das tuplas em objetos. Também oferece a possibilidade da manipulação dos dados por meio da linguagem SQL (*structured query language*).

Modelo de banco de dados semiestruturado

Os bancos de dados desenvolvidos no modelo semiestruturado são pouco utilizados se comparados com as arquiteturas anteriores. Por não possuírem uma estrutura rígida, possibilitam perda da integridade dos dados. São aqueles que não utilizam o modelo relacional integralmente, mas contam com marcações para organização dos dados. Bancos de dados desse modelo não possuem linguagem de consulta padrão; por isso, deve ser observada a linguagem adequada para cada caso.

Modelos de banco de dados NoSQL (ou não relacionais)

Em 1998, surgiu o termo **NoSQL**, a partir de uma solução de banco de dados ainda baseada na arquitetura do modelo relacional e que não oferecia, para o seu usuário, uma interface SQL. Posteriormente, o termo tornou-se uma abreviação para *not only SQL* (“não apenas SQL”), oferecendo uma solução alternativa para os cenários em que o consagrado modelo relacional não apresentava o desempenho adequado. O objetivo das soluções NoSQL é fornecer uma solução com maior flexibilidade na estruturação de um banco de dados, quando houver essa necessidade. Portanto, seu propósito não é o de substituir o modelo relacional de dados.

Podemos categorizar os modelos de dados NoSQL pela estrutura com que trabalham: família de colunas, documentos, chave-valor e grafos, descritos a seguir.

- **Família de colunas:** armazena dados mapeados por chaves, e os dados agrupados formam uma espécie de mapa. Exemplos: BigTable e Cassandra.

- **Documentos:** armazena documentos como um valor, que possui uma chave para identificação, como no modelo por chave. Exemplos: MongoDB e CouchDB.
- **Chave-valor:** guarda uma informação ou valor e cria uma referência para acessá-lo, a chave. Exemplo: Amazon Dynamo.
- **Grafos:** trata-se da criação de nodos, ou entidades, ou nós. As arestas do grafo são as representações das relações. Exemplos: InfoGrid e Neo4j.

Assim como os bancos de dados semiestruturados, os bancos de dados NoSQL não apresentam linguagem de consulta padrão, até pelo fato de terem estruturas diferentes entre si.

Esquema de dados

Podemos afirmar que, em um modelo de dados, um **esquema** é uma descrição em uma linguagem formal da estrutura do banco de dados. De acordo com Alves (2013, p. 32), “[...] para construção de um esquema, devem ser desprezadas todas as especificações dos campos das tabelas, listando apenas os seus nomes”. Na Figura 5, podemos observar um exemplo de um diagrama para o esquema de um banco de dados. É possível notar a identificação das chaves primárias de cada tabela, observando o sublinhado que foi colocado no nome dos referidos campos.

Tabela de categorias

<u>CodigoCategoria</u>	DescricaoCategoria
------------------------	--------------------

Tabela de categorias_produtos

<u>CodigoCategoria</u>	<u>CodigoProduto</u>
------------------------	----------------------

Tabela de produtos

<u>CodigoProduto</u>	NomeProduto	Preco
----------------------	-------------	-------

Tabela de fornecedores_produtos

<u>CodigoFornecedor</u>	<u>CodigoProduto</u>
-------------------------	----------------------

Tabela de fornecedores

<u>CodigoFornecedor</u>	NomeFornecedor	Endereco	Bairro	Cidade	Estado
-------------------------	----------------	----------	--------	--------	--------

Figura 5. Exemplo de um esquema de banco de dados.

Fonte: Alves (2013, p. 33).

A arquitetura de três esquemas é também conhecida como arquitetura ANSI/SPARC, em que ANSI se refere ao American National Standards Institute, e SPARC se refere ao Standards Planning and Requirements Committee. Ela tem o objetivo de separar o usuário da aplicação do banco de dados físico, por meio de três diferentes níveis de esquemas (ELMASRI; NAVATHE, 2011), descritos a seguir.

- **Interno:** a partir de um esquema interno, o nível interno descreve a estrutura de armazenamento físico do banco de dados. Assim, é possível descrever os detalhes complexos do armazenamento, bem como os caminhos de acesso ao banco de dados.
- **Externo:** ao abranger os esquemas externos ou visões de usuários, no nível externo, o esquema externo descreve determinada parte do banco de dados que um dado grupo de usuários tenha interesse. Além disso, oculta o restante do banco de dados desse grupo. Nesse nível, ao utilizar um modelo de dados representacional, cada esquema externo é tipicamente implementado. Isso é feito, possivelmente, com base em um projeto de esquema externo em um modelo de dados de alto nível.
- **Conceitual:** por meio de um esquema conceitual, descreve a estrutura de todo o banco de dados para seus usuários. Por ele, são ocultados os detalhes das estruturas de armazenamento físico, concentrando-se na descrição de entidades, tipos de dados, conexões, operações de usuários e restrições. Quando o sistema de banco de dados é implementado, geralmente, um modelo de dados representacional é usado para descrever o esquema conceitual. Dessa forma, é fundamentado em um projeto de esquema conceitual, em um modelo de dados de alto nível.

Tipos de dados

Com o passar do tempo, surgiram novos tipos de dados, e, conseqüentemente, os SGBDs se adaptaram para aceitá-los sem a necessidade de trabalhar com conversões o tempo todo. Atualmente, quanto aos tipos de dados, os SGBDs podem ser classificados em dados clássicos, textos/documentos, dados multimídia e dados geográficos, categorias descritas a seguir.

- **Dados clássicos:** comportam apenas valores numéricos e textuais.
- **Textos/documentos:** armazenam os dados em formato de texto ou documentos. Normalmente utilizam arquivos com a extensão XML.
- **Dados multimídia:** armazenam imagens, arquivos em PDF, entre outros.
- **Dados geográficos:** reconhecem pontos, linhas e polígonos, que são utilizados para criar representações do espaço.

Níveis de abstração

Na arquitetura de banco de dados, temos várias categorias de modelos de dados. Podemos classificar cada modelo de acordo com os tipos de conceitos utilizados para descrever sua estrutura. Os modelos de dados conceituais, ou modelos de alto nível, possuem conceitos que descrevem os dados como são percebidos pelos usuários. Os **modelos de dados conceituais** usam conceitos como entidades, atributos e relacionamentos. Uma **entidade** corresponde a um objeto do mundo real descrito no banco de dados. Um **atributo** corresponde a uma propriedade que auxilia na descrição de uma entidade. O **relacionamento** entre duas ou mais entidades representa uma associação entre elas.

Os **modelos de dados físicos**, ou modelos de baixo nível, contêm conceitos que descrevem os dados, detalhadamente, como estão armazenados no banco de dados. Esses modelos descrevem os dados pela representação do formato dos registros, da ordem e da disposição dos registros e das suas rotas de acesso, que são estruturas que tornam as pesquisas por registros no banco de dados mais rápidas e eficientes. Os conceitos obtidos dos modelos de dados de baixo nível geralmente são mais significativos para os especialistas em tecnologia da informação do que para os usuários finais das aplicações.

Entre esses dois níveis extremos, temos uma classe de modelos de dados chamada de **modelos representacionais ou de implementação**, que descreve conceitos que ocultam alguns detalhes de como os dados estão organizados e armazenados no banco de dados. Estes, porém, podem ser interpretados pelos usuários finais dos aplicativos. Atualmente, os modelos de dados representacionais são os mais usados comercialmente pelos SGBDs mais tradicionais. Essa classe inclui o modelo de dados relacional, utilizado mais popularmente nos projetos de bancos de dados, e também os modelos hierárquicos e os modelos de rede, considerados modelos de dados legados, que já foram muito utilizados no passado.

2 Principais sistemas de gerenciamento de banco de dados disponíveis no mercado

Nesta seção, serão apresentados alguns exemplos e características de SGBDs comerciais e gratuitos que estão disponíveis no mercado. Para isso, eles serão separados em SGBDs relacionais e SGBDs não relacionais.

Sistemas de gerenciamento de banco de dados relacionais

Veja a seguir alguns dos principais SGBDs relacionais disponíveis no mercado.

Oracle

A Oracle é uma empresa que surgiu nos anos 1980. Hoje, é uma das maiores do mercado da tecnologia da informação e é proprietária de diversos recursos conhecidos, como o banco da Oracle e o pacote de desenvolvimento Java. O SGBD é um dos produtos mais conhecidos da empresa e é utilizado mundialmente. É uma ferramenta robusta, constantemente atualizada e aperfeiçoada para atender às demandas do mercado. Esse SGBD é desenvolvido com foco no mercado empresarial, e seus recursos atendem a empresas de pequeno e grande porte. Adota o modelo de dados relacional e oferece a possibilidade de manipulação de seus dados por meio da linguagem procedural/SQL. A Oracle trabalha com o lançamento de diferentes versões, com características diferentes e adequadas a necessidades diversas, sob a concessão de diferentes licenças.

MySQL

O banco de dados MySQL também é um dos mais populares em todo o mundo. É uma tecnologia de código aberto, que permite que os usuários e as organizações modifiquem o SGBD de acordo com as suas necessidades. O MySQL foi criado na Suécia, por volta da década de 1980. Em 2009, foi vendido para a Oracle por cerca de um bilhão de dólares. É um banco de dados que pode ser utilizado em diferentes plataformas. O seu uso é simples, o que o torna muito conhecido.

Microsoft SQL Server

A primeira versão do SQL Server, que é o SGBD relacional da Microsoft, foi lançada em 1988 como componente do sistema operacional Windows NT. Somente em versões posteriores foi comercializado como um produto para aquisição individual. A grande diferença em relação aos outros SGBDs citados é que o SQL Server possibilita a realização de consultas e a manipulação dos dados por meio de linguagens de programação, como a C# e a Visual Basic. Ainda, é possível trabalhar com a linguagem SQL. O SQL Server também pode ser integrado aos recursos do SQL Server Management Studio e à extensão para linguagem T-SQL. Apesar de ser uma solução paga, é um dos bancos de dados mais utilizados no mundo, devido à sua qualidade e robustez.

PostgreSQL

O PostgreSQL é um banco de dados de modelo objeto-relacional, de código aberto, lançado em 1989. É um dos mais utilizados na atualidade, especialmente para o desenvolvimento de sistemas web e de soluções para demandas de negócios. Esse SGBD é considerado, por muitos, o melhor banco de dados de código aberto disponível; por isso, é adotado por diversas organizações públicas, no Brasil e no exterior.

MariaDB

O MariaDB é um banco que surgiu com base no MySQL e ganhou popularidade rapidamente. Quando a Oracle adquiriu o MySQL, decidiu trabalhar em um novo SGBD para aprimorar as características do MySQL. Então, em 2009, foi lançado o MariaDB. Esse SGBD possui boas perspectivas de crescimento no mercado, principalmente devido à sua simplicidade de uso e à preocupação com a segurança dos dados.

Sistemas de gerenciamento de banco de dados não relacionais

Veja a seguir alguns dos principais SGBDs não relacionais disponíveis no mercado.

MongoDB

É um banco de dados relativamente recente. Sua primeira versão foi lançada em 2009, e seu uso vem se expandindo desde então. O MongoDB é um banco de dados NoSQL, de código aberto, que trabalha com o modelo de dados orientado a documentos. Foi desenvolvido para unir características dos bancos de dados relacionais e NoSQL. Como resultado dessa mistura de abordagens, o MongoDB apresenta ganhos de agilidade com o uso de esquemas flexíveis e facilita a escalabilidade horizontal, recursos pouco explorados por outros SGBDs, como MySQL e PostgreSQL. O uso do MongoDB tem aumentado, o que levou a uma expansão conjunta da linguagem JavaScript, utilizada por esse SGBD.

Redis

O Redis é um banco de dados que também trabalha com o modelo de dados NoSQL. A estruturação de seus dados é feita com base no modelo chave-valor. É capaz de armazenar textos, listas, conjuntos, entre outros itens. Assim como outros dados NoSQL, o interesse das empresas em seu uso vem crescendo significativamente.

Cassandra

O Cassandra, assim como o MongoDB, é um banco de dados NoSQL. Foi criado pelo Facebook, que tornou o projeto de código aberto em 2008. Hoje é mantido pelos membros da fundação Apache. Esse SGBD também adota a linguagem JavaScript. Sua grande vantagem frente a outros SGBDs é a capacidade de escalabilidade sem causar perdas de performance.

Os usuários do sistema de banco de dados

Para um sistema de banco de dados aplicado em um projeto de média ou alta complexidade, existem muitos profissionais (usuários) envolvidos desde a fase de criação do projeto até a fase de manutenção propriamente dita. De acordo com Cardoso e Cardoso (2013, p. 24), “[...] no ambiente de um sistema de banco de dados é importante definir os usuários e suas funções específicas, sendo que a cada um cabe uma tarefa diferenciada”.

Para Alves (2013), os usuários podem ser classificados conforme descrito a seguir.

- **Projetistas de banco de dados:** profissionais cuja responsabilidade é identificar todos os dados que devem ser armazenados no banco de dados, assim como determinar a sua estrutura por meio da especificação de um modelo de dados. Para se chegar a essa fase, é necessário um levantamento das necessidades dos usuários finais. O projeto final do banco de dados deve atender a essas necessidades, sendo o mais completo possível.
- **Analistas de sistemas e programadores:** os analistas de sistemas são responsáveis pela especificação da estrutura e das funcionalidades do aplicativo, depois de se determinarem as necessidades dos usuários. Então, os programadores, com base no modelo de dados definido pelo projetista de banco de dados e nas especificações dos analistas, desenvolvem a aplicação propriamente dita, criando formulários de interface com o usuário e códigos que executam tarefas importantes. São eles também os responsáveis pela documentação e manutenção do sistema. Para o desenvolvimento, é imprescindível que analistas e programadores estejam familiarizados com o SGBD e a linguagem de programação adotados.
- **Administradores de banco de dados:** são os profissionais responsáveis pela administração dos recursos oferecidos pelo SGBD, também conhecidos como *database administrator* (DBA). Entre as suas principais tarefas, estão: definir novos usuários do sistema, determinar níveis de acesso a cada usuário, monitorar o uso do sistema e efetuar manutenções preventivas (como cópia de segurança e restauração de bancos de dados). O DBA é responsável por planejar, documentar e gerenciar os recursos de informação do negócio em que atua.
- **Usuários finais:** são as pessoas que efetivamente fazem uso do aplicativo e, por conseguinte, do banco de dados. Este foi elaborado principalmente para atender às exigências que esses usuários apresentaram aos projetistas e analistas.

Podemos concluir que os usuários podem ser diferenciados ou agrupados conforme seus papéis e suas expectativas de interação com o sistema de banco de dados.